APEX TRADER

A Stock and Cryptocurrency Trading Database Management System

with an In-Database Matching Engine and a Double-Entry Ledger

BCSE302P — Database Systems Lab

Submitted by

24BCE2863 Kumar Paritosh

24BCE2865 Samrender Singh Saini

24BCE2886 Vartika Chaudhary

Under the guidance of

Prof. Nithya N S

School of Computer Science and Engineering
Vellore Institute of Technology, Vellore

PostgreSQL 16 · Python 3.13 · FastAPI

38 tables · 84 routines · 24 RLS policies · 150 SQL assertions · 118 API checks · 51 browser checks

What this presentation covers

The guideline sequence, with the evidence for each claim

Design

- Abstract and problem statement
- Objectives and scope
- System architecture and workflow
- ER diagram
- Relational schema and normalization

Implementation

- CREATE TABLE and constraints
- Sample data and volumes
- SQL queries and their outputs
- The order round trip
- Operations the database refuses

Advanced + proof

- Procedures, triggers, views
- Transaction control
- Row Level Security
- Partitioning and indexing
- Surveillance, screenshots, testing

Every output in this deck is a captured transcript from the running database. Nothing on the following slides was typed by hand.

Abstract and Problem Statement

Apex Trader is a database management system for a stock and cryptocurrency trading venue. The limit order book is a relation, matching is a stored procedure, settlement is a journal entry, and all three execute inside one database transaction. The database is the exchange — not a store behind one.

The problem

- Who owns what, at what price it changed hands, and whether the trade was allowed are three answers that depend on each other.
- They must all hold while many users trade on the same instrument at once.
- Risk in Python, matching in Python, then writing balances cannot be made atomic without a global lock or a distributed transaction.
- A stored balance can silently diverge from the history that produced it, and nothing in the schema says so.

The approach

- Matching, settlement and risk run in ONE transaction, in PL/pgSQL.
- No balance is stored. Every balance is derived from an append-only double-entry journal.
- A DEFERRABLE INITIALLY DEFERRED constraint verifies at COMMIT that each entry sums to zero, so value can move but never be created.
- Tenant isolation by Row Level Security, not by remembering a WHERE clause.

Objectives

Eight objectives, and how each was realised

Model a complete trading venue relationally — 38 base tables in BCNF across 12 bounded-context schemas

Implement order matching inside the database — price-time priority, 6 order types, 4 time-in-force policies

Guarantee accounting integrity by construction — append-only journal, derived balances, deferred zero-sum constraint

Enforce risk before a trade, not after — a pre-trade gate that runs inside the order transaction

Isolate tenants in the database layer — 24 Row Level Security policies over 16 tables

Detect market abuse using analytical SQL — wash trading, spoofing, momentum ignition, volume anomaly

Support multiple currencies without loss of integrity — 10 currencies, conversion is a four-legged journal entry

Demonstrate correctness rather than assert it — 150 SQL assertions, 118 API checks, 51 browser checks, 18 invariants

Scope delivered

38 + 30	84	20	12 + 2	348	24	28
tables + partitions	routines	triggers	views + matviews	indexes	RLS policies	migrations

Two asset classes on one engine: equities and ETFs in a 390-minute session, eight crypto pairs trading 24×7. What differs is data on the instrument, not code in the engine.

Scope and Delimitations

What the system does, and what it deliberately does not

In scope

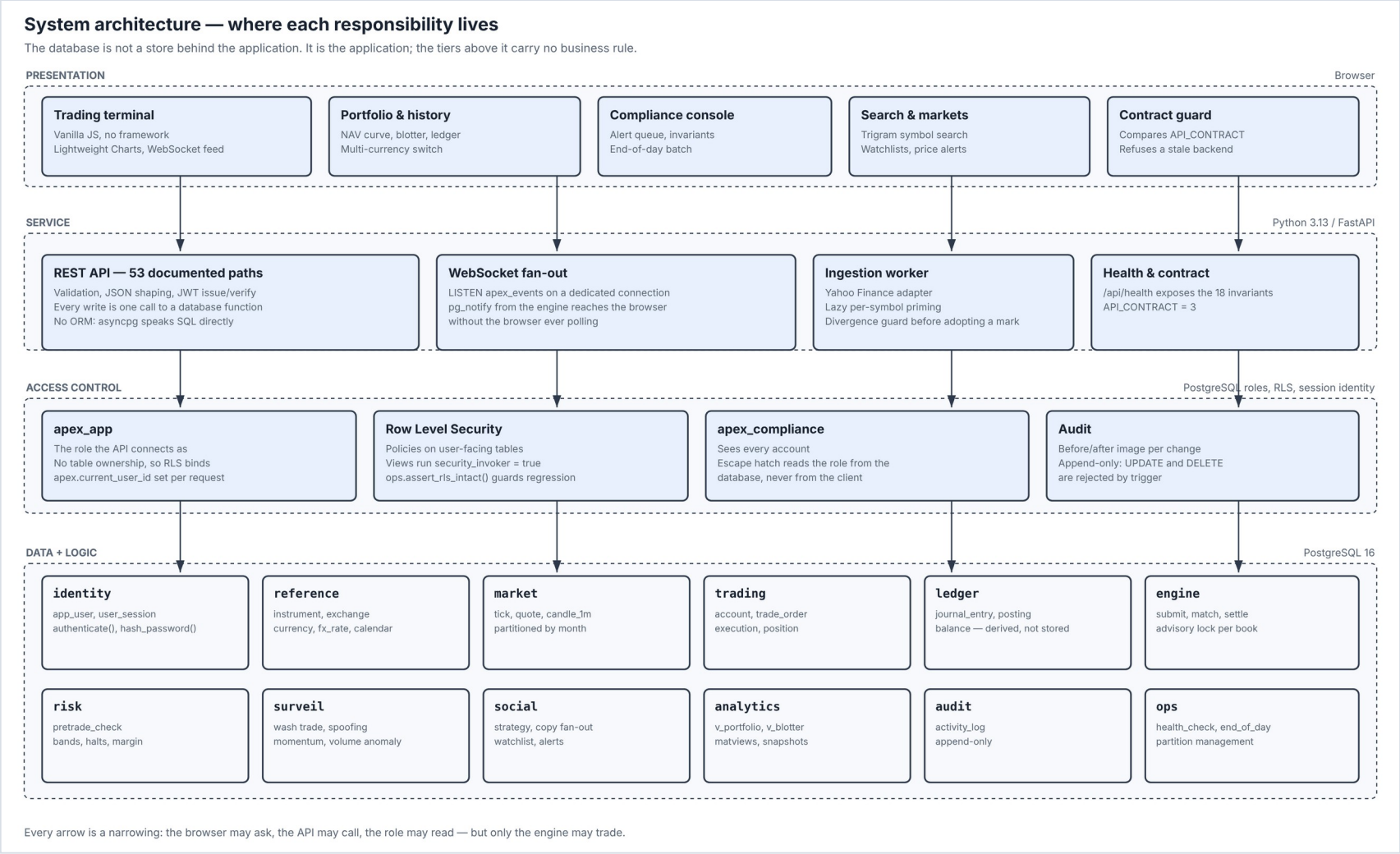
- Continuous auction with price-time priority on a per-instrument book.
- Market, limit, stop, stop-limit, trailing-stop and iceberg orders.
- DAY, GTC, IOC and FOK time-in-force policies.
- Double-entry settlement with derived balances in 10 currencies.
- Pre-trade risk: buying power, short-sale gate, notional cap, concentration, rate limit, daily loss, price band, halts.
- Surveillance, audit trail, copy trading, watchlists and price alerts.
- Minute bars, arbitrary chart intervals, and technical indicators in SQL.

Out of scope, and why

- Opening and closing auctions — the session model supports them, but the auction algorithm is a separate piece of work.
- Clearing-house interposition and T+1 settlement — the journal already carries occurred_at; a settlement date completes it.
- Corporate actions applied to historical prices — modelled, not back-applied.
- Options and futures — the ledger needs a third asset class.
- Full order-book depth beyond top of book over the wire.
- Replication and clustering — single-node by design for this submission.

System Architecture

Four tiers — and the business logic lives in exactly one of them



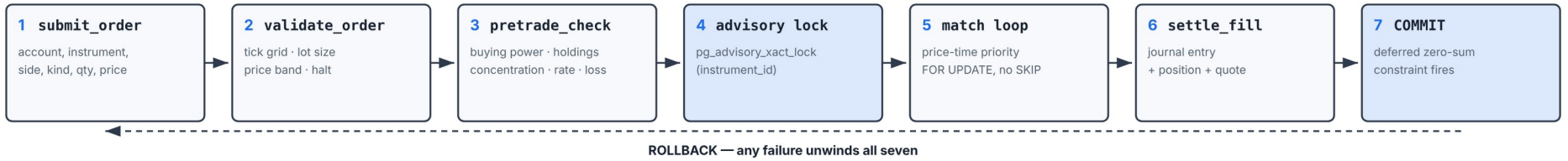
The service tier validates request shape and emits JSON. It calls one database function per write and contains no rule that could disagree with the database.

Workflow — the life of an order

Seven stages, one transaction. Any failure unwinds all seven.

Order lifecycle — one transaction, seven stages

Any stage that raises rolls back every earlier one. There is no partially applied order.



What stage 5 actually does

```
FOR v_resting IN
    SELECT order_id, limit_price, leaves_qty
      FROM trading.trade_order
     WHERE instrument_id = p_instrument_id
       AND side = v_opposite
       AND status IN ('working','partially_filled')
     ORDER BY CASE WHEN p_side='buy'  THEN limit_price END ASC,
              CASE WHEN p_side='sell' THEN limit_price END DESC,
              sequence_no              -- time priority
    FOR UPDATE                        -- never SKIP LOCKED
LOOP
    v_fill := LEAST(v_remaining, v_resting.leaves_qty);
    PERFORM engine.settle_fill(...);
END LOOP;
```

Why FOR UPDATE and not FOR UPDATE SKIP LOCKED

SKIP LOCKED is the usual advice for queue draining, and it is exactly wrong here.

If order A at 174.87 is locked by another session, SKIP LOCKED steps over it and fills order B at the same price — B arrived later, so the exchange has just broken its own time-priority promise. The trade is not incorrect in the accounting sense; it is incorrect in the FAIRNESS sense, which is the one a venue is actually selling.

So the session waits. Throughput across one instrument is the price of a guarantee that the earliest order at a price always fills first. Different instruments still match in parallel, because the advisory lock is keyed on instrument_id.

Concurrency and isolation

- READ COMMITTED is sufficient because correctness does not rest on the isolation level — it rests on the lock and the constraints.
- pg_advisory_xact_lock(instrument_id) serialises matching per instrument and releases at COMMIT or ROLLBACK with no cleanup code.
- FOR UPDATE takes a row lock on each resting order considered, so two matchers cannot both decide the same order has quantity left.
- The deferred constraint is evaluated once per transaction at COMMIT, after every leg of every entry has been written.
- Verified empirically: 12 concurrent sessions, 300 orders, 3,000 shares demanded against 3,000 available — filled exactly 3,000,
- zero overfills, zero shares or cash created or destroyed.

Stage 4 is the serialisation point and stage 7 is where the ledger invariant is checked. Between them, the engine may fill against many resting orders.

Database Management System

Three capabilities on which the design depends

PostgreSQL 16. Version 15 is the minimum, and the reason is on this slide.

Deferrable constraint triggers

The legs of a journal entry are inserted one row at a time, so the entry is legitimately unbalanced DURING the transaction. The check must run at COMMIT. MySQL has no DEFERRABLE constraint, so the rule would move into application code — precisely the place this design removes it from.

Row Level Security

Tenant isolation becomes a property of the data rather than of the queries that read it. A query that omits its account predicate returns an empty set instead of another trader’s rows. MySQL offers no equivalent.

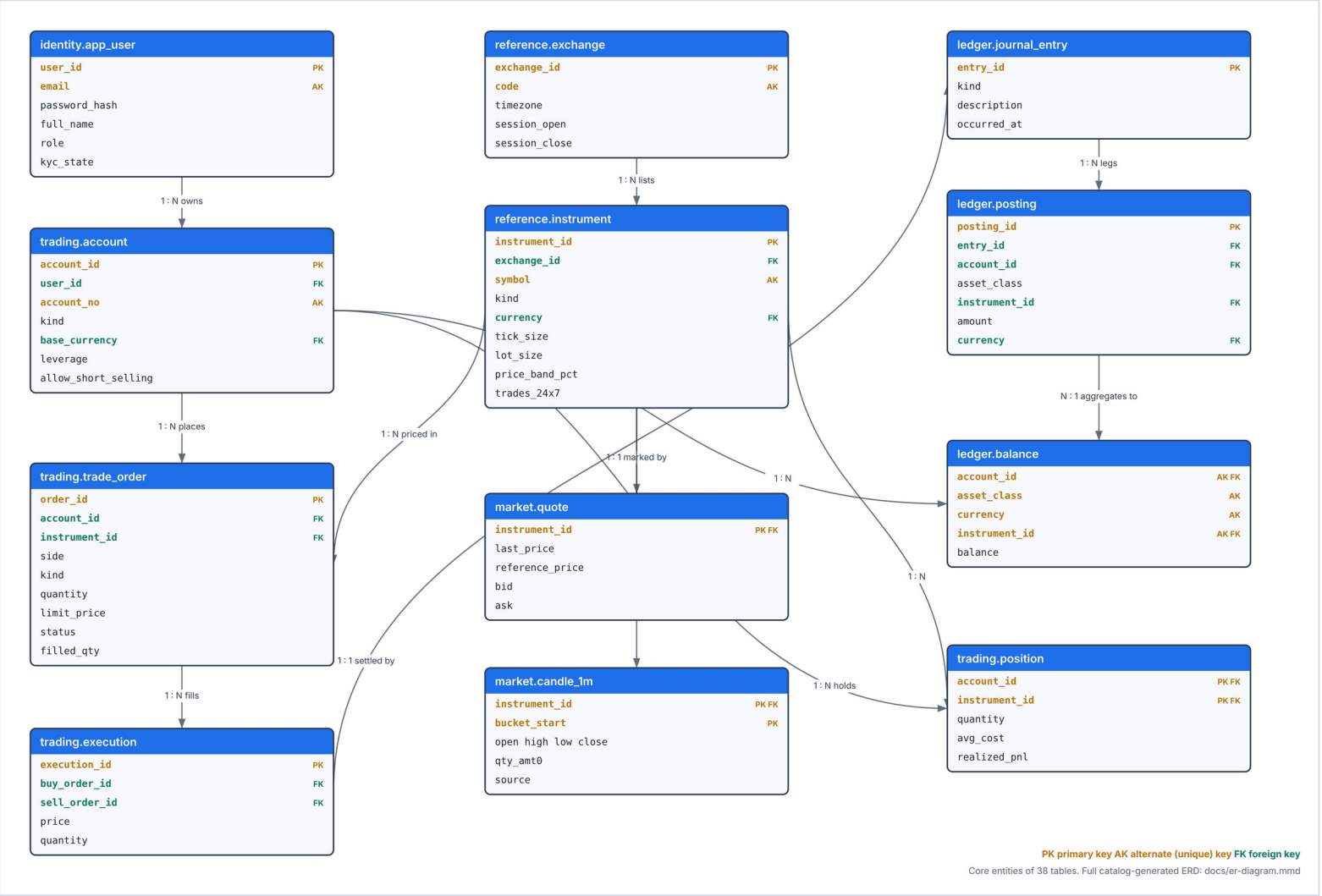
NULLS NOT DISTINCT (PostgreSQL 15+)

A cash balance has no instrument. Under standard SQL two NULLs are distinct, so a conventional unique key would let one account hold two cash balances in the same currency. This one clause sets the minimum version for the project.

```
CREATE UNIQUE INDEX uq_balance_slot ON ledger.balance
    (account_id, asset_class, currency, instrument_id) NULLS NOT DISTINCT;
```

ER Diagram — core entities

PK primary key · AK alternate (unique) key · FK foreign key



Twelve of the 38 base tables: the ones a reader must follow to see how an order becomes a settled position. The full catalog-generated ERD is docs/er-diagram.mmd.

Relational Schema

Twelve bounded-context schemas, not one flat namespace

Schema	Responsibility	Representative objects
`identity`	Users, credentials, sessions	app_user, user_session, login_attempt
`reference`	Security master and calendars	instrument, exchange, currency, fx_rate
`market`	Market data	tick, quote, candle_1m (partitioned by month)
`trading`	Accounts, orders, fills, positions	account, trade_order, execution, position
`ledger`	The book of record	journal_entry, posting, balance
`engine`	Matching and settlement	submit_order, match_order, settle_fill
`risk`	Pre-trade control	pretrade_check, trading_halt, price_alert
`surveil`	Market-abuse detection	rule, alert
`social`	Copy trading and watchlists	strategy, copy_subscription, watchlist
`analytics`	Derived read models	v_portfolio, v_blotter, account_snapshot
`audit`	Immutable change log	activity_log
`ops`	Operations	health_check, schema_migration, partitions

Schemas give readable qualified names, a per-context GRANT surface, and a natural attachment point for security policies. The dependency direction is explicit: engine depends on ledger and risk, never the reverse.

Normalization

Boyce-Codd Normal Form, and the three deliberate exceptions

Normalization — the chain, and the three exceptions

Every step is stated as a functional dependency, because the claim without the dependency is an assertion, not an analysis.

1NF
Atomic attributes
Every attribute holds one value. metadata is JSONB: provenance, never a queried attribute.

→

2NF
No partial dependency
Only three relations have a composite key. In each, every attribute needs the whole key.

→

3NF
No transitive dependency
instrument.currency is not derived from exchange: an ADR can trade in another currency.

→

BCNF
Every determinant is a key
All 38 base tables. No relation has a non-trivial FD whose left side is not a superkey.

Three deliberate denormalizations — each one a cache, each one verified

Relation	Derived from	Maintained by	Verified by
`ledger.balance`	SUM over ledger.posting	trigger trg_posting_updates_balance	ledger.verify_integrity()
`trading.position`	Security postings per instrument	trigger trg_posting_syncs_position	positions_match_ledger
`market.quote`	Latest market.tick / best bid and ask	engine.refresh_top_of_book()	no_crossed_books

The one place normal form was knowingly broken, and why that is safe

A balance column that can drift from its own history is the classic argument against storing it at all. Here the column is never the source of truth: it is a cache, the journal is the truth, and ops.health_check() re-derives every balance from the journal and reports any slot that disagrees. The denormalization buys a constant-time read of buying power on the order path; the invariant buys back the correctness it cost.

A denormalization that cannot be checked is a defect awaiting discovery. Each of the three is a cache with a trigger that maintains it and an invariant that verifies it.

Database Implementation — CREATE TABLE

trading.trade_order

```
CREATE TABLE trading.trade_order (
    order_id        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    -- Idempotency key: a retried POST cannot create a second order.
    client_order_id UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE,
    -- The "time" in price-TIME priority. Timestamps can tie and can
    -- move backwards under NTP; a sequence can do neither.
    seq_no          BIGINT NOT NULL DEFAULT nextval('engine.order_seq'),

    account_id      BIGINT  NOT NULL REFERENCES trading.account(account_id),
    instrument_id   INTEGER NOT NULL REFERENCES reference.instrument(...),
    side            trading.order_side    NOT NULL,
    kind            trading.order_kind    NOT NULL,
    tif             trading.time_in_force NOT NULL DEFAULT 'day',

    quantity        qty_amt  NOT NULL,
    limit_price     NUMERIC(20,8) CHECK (limit_price IS NULL OR > 0),
    filled_qty      qty_amt0 NOT NULL DEFAULT 0,
    -- GENERATED: remaining size is arithmetic, not state.
    leaves_qty      NUMERIC(20,8) GENERATED ALWAYS AS
                    (quantity - filled_qty) STORED,
    status          trading.order_status NOT NULL DEFAULT 'pending',

    CONSTRAINT limit_requires_price CHECK (
        kind NOT IN ('limit','stop_limit') OR limit_price IS NOT NULL),
    CONSTRAINT no_overfill CHECK (filled_qty <= quantity));
```

Constraints on show

- PRIMARY KEY identity column
- UNIQUE client_order_id
- FOREIGN KEY account, instrument
- NOT NULL on every required column
- CHECK two table-level rules
- GENERATED leaves_qty

Why a sequence, not a clock

- Two orders placed in the same millisecond tie on a timestamp.
- An NTP correction can move a clock backwards and reorder them.
- engine.order_seq gives a total order that nothing outside the database can disturb — which is what fairness means here.

Database Implementation — the ledger

ledger.posting, and the constraint that makes double entry a guarantee

```
CREATE TABLE ledger.posting (
    posting_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    entry_id      BIGINT NOT NULL REFERENCES ledger.journal_entry(entry_id)
                                  ON DELETE CASCADE,
    account_id    BIGINT NOT NULL REFERENCES trading.account(account_id),
    asset_class   ledger.asset_class NOT NULL,
    instrument_id INTEGER REFERENCES reference.instrument(instrument_id),
    currency      currency_code REFERENCES reference.currency(code),
    -- Positive = value flows INTO this account, negative = OUT of it.
    amount        money_amt NOT NULL CHECK (amount <> 0),

    CONSTRAINT asset_instrument_agreement CHECK (
        (asset_class = 'cash'     AND instrument_id IS NULL)
     OR (asset_class = 'security' AND instrument_id IS NOT NULL)));

CREATE CONSTRAINT TRIGGER trg_posting_balanced
    AFTER INSERT OR UPDATE OR DELETE ON ledger.posting
    DEFERRABLE INITIALLY DEFERRED
    FOR EACH ROW EXECUTE FUNCTION ledger.assert_entry_balanced();
```

The sign convention

- A posting is one leg of an entry.
- Positive flows in, negative flows out.
- An entry balances when its legs sum to zero — independently for cash and for every security.

Why DEFERRABLE

- Legs are inserted one row at a time, so the entry is legitimately unbalanced while that happens.
- A per-statement check would reject every correct entry.
- Deferring it to COMMIT is what lets the rule be absolute.

Database Implementation — partitioned market data

market.candle_1m

```
CREATE TABLE market.candle_1m (
    instrument_id INTEGER     NOT NULL REFERENCES reference.instrument(...),
    bucket_start  TIMESTAMPTZ NOT NULL,
    open          price_amt   NOT NULL,
    high          price_amt   NOT NULL,
    low           price_amt   NOT NULL,
    close         price_amt   NOT NULL,
    qty_amt0      qty_amt0    NOT NULL DEFAULT 0,
    trade_count   INTEGER     NOT NULL DEFAULT 0 CHECK (trade_count >= 0),
    vwap          NUMERIC(20,8),
    PRIMARY KEY (instrument_id, bucket_start),

    -- OHLC has an internal arithmetic truth. A feed defect that
    -- violates it is rejected at write time, not discovered later
    -- as an impossible candlestick on a chart.
    CONSTRAINT ohlc_coherent CHECK (
        high >= low AND high >= open AND high >= close
                    AND low  <= open AND low  <= close),
    CONSTRAINT bucket_aligned CHECK (
        date_trunc('minute', bucket_start) = bucket_start)
) PARTITION BY RANGE (bucket_start);
```

Domains do the repeating

- price_amt NUMERIC(20,8) CHECK (> 0)
- qty_amt0 NUMERIC(20,8) CHECK (>= 0)
- The constraint travels with the type, so it cannot be forgotten on a column that uses it.

Partitioning buys two things

- The planner prunes whole months at plan time — measured on a later slide.
- An expired month is detached in constant time instead of deleted row by row.
- ops.ensure_time_partitions keeps a runway of future partitions ahead of the data.

Constraints — counted from the catalogue, not claimed

540	71	57	37	11	1	1
NOT NULL	CHECK	FOREIGN KEY	PRIMARY KEY	UNIQUE	EXCLUDE	CONSTRAINT TRIGGER

```
        kind         | declared
---------------------+----------
 NOT NULL            |      540
 CHECK               |       71
 FOREIGN KEY         |       57
 PRIMARY KEY         |       37
 UNIQUE              |       11
 EXCLUDE             |        1
 CONSTRAINT TRIGGER  |        1
(7 rows)
```

The EXCLUDE constraint

- no_overlapping_halts, over GiST with btree_gist:

  two trading halts on one instrument cannot overlap in time.
- A rule that would otherwise be a race condition in application code.

The CONSTRAINT trigger

- trg_posting_balanced, DEFERRABLE INITIALLY DEFERRED:

  every journal entry sums to zero, verified at COMMIT.
- This single object is what makes the ledger provable.

Sample Data

Generated so that it is internally consistent, by ordinary SQL

```
INSERT INTO reference.currency (code, name, symbol, minor_units) VALUES
  ('USD','US Dollar','$',2), ('INR','Indian Rupee','₹',2),
  ('EUR','Euro','€',2),      ('GBP','Pound Sterling','£',2)
ON CONFLICT (code) DO NOTHING;

INSERT INTO reference.instrument (symbol, exchange_id, name, sector,
                                  currency, tick_size, lot_size,
                                  price_band_pct, is_shortable)
VALUES ('AAPL', 1, 'Apple Inc.', 'Technology', 'USD', 0.01, 1, 20.0, TRUE);

-- The password is never stored: pgcrypto hashes it with a per-row salt.
INSERT INTO identity.app_user (email, password_hash, full_name, role, kyc_state)
VALUES ('samrender@apex.io', identity.hash_password('Apex@2026'),
        'Samrender Singh Saini', 'trader', 'verified');

-- Funding an account is a LEDGER EVENT, not an UPDATE of a balance
-- column: the money is moved from the system bank account.
SELECT ledger.post_entry('deposit', 'Opening balance', ARRAY[
    ROW(1,   'cash', NULL, -500000.00, 'funding')::ledger.leg,
    ROW(375, 'cash', NULL,  500000.00, 'funding')::ledger.leg]);
```

```
        relation        | count
------------------------+--------
 audit.activity_log     |    704
 identity.app_user      |      8
 ledger.balance         |    148
 ledger.journal_entry   |    257
 ledger.posting         |   1130
 market.candle_1m       | 442080
 market.tick            |      7
 reference.currency     |     10
 reference.fx_rate      |    414
 reference.instrument   |     24
 trading.account        |     13
 trading.execution      |    174
 trading.position       |    123
 trading.trade_order    |    599
(14 rows)
```

Forty-five trading sessions. The market history is self-consistent: candles agree with the ticks they were aggregated from, every execution has a balanced journal entry, and every position equals the sum of its postings.

SQL Queries and Outputs — joins

A portfolio cannot be answered from one table

```
SELECT split_part(u.full_name,' ',1) AS trader, a.account_no, i.symbol,
       round(p.quantity,2) AS qty, round(p.avg_cost,2) AS avg_cost,
       round(q.last_price,2) AS last_px,
       round((q.last_price - p.avg_cost) * p.quantity, 2) AS unrealised
  FROM trading.position  p
  JOIN trading.account   a ON a.account_id    = p.account_id
  JOIN identity.app_user u ON u.user_id       = a.user_id
  JOIN reference.instrument i ON i.instrument_id = p.instrument_id
  JOIN market.quote      q ON q.instrument_id = p.instrument_id
 WHERE p.quantity <> 0 ORDER BY unrealised DESC LIMIT 8;
```

```
  trader   | account_no | symbol  |  qty   | avg_cost | last_px | unrealised
-----------+------------+---------+--------+----------+---------+------------
 Sam       | APX-000345 | MSFT    |  48.00 |   505.24 |  751.22 |   11807.04
 Samrender | APX-000342 | TSLA    |  58.00 |   383.71 |  561.33 |   10302.07
 Vartika   | APX-000343 | QQQ     |  53.00 |   493.49 |  673.05 |    9516.93
 Samrender | APX-000342 | SOL-USD |  62.26 |   192.75 |  313.62 |    7524.98
 Ginni     | APX-000346 | TSLA    |  21.00 |   235.88 |  561.33 |    6834.45
 Kumar     | APX-000344 | QQQ     |  30.00 |   464.38 |  673.05 |    6260.04
 Vartika   | APX-000343 | SPY     |  87.00 |   555.51 |  617.20 |    5366.84
 Samrender | APX-000342 | V       | 171.00 |   281.71 |  298.58 |    2883.98
(8 rows)
```

Five tables from four bounded contexts. The join is the reason the data is normalised: the average cost lives in exactly one place and the live price in exactly one other.

SQL Queries and Outputs — aggregates and subqueries

GROUP BY, HAVING and the aggregate FILTER clause

```
SELECT i.symbol, count(*) AS orders,
       count(*) FILTER (WHERE o.side='buy')  AS buys,
       count(*) FILTER (WHERE o.side='sell') AS sells,
       round(sum(o.filled_qty))         AS filled,
       round(avg(o.avg_fill_price), 2)  AS avg_px
  FROM trading.trade_order o
  JOIN reference.instrument i USING (instrument_id)
 GROUP BY i.symbol HAVING count(*) >= 10
 ORDER BY orders DESC LIMIT 8;
```

```
 symbol | orders | buys | sells | filled | avg_px
--------+--------+------+-------+--------+--------
 DIS    |     54 |   33 |    21 |   2400 |  98.74
 INTC   |     48 |   24 |    24 |   9748 |  43.05
 NVDA   |     36 |   18 |    18 |   2518 |  84.89
 AMZN   |     35 |   18 |    17 |   1112 | 177.28
 AAPL   |     35 |   20 |    15 |    934 | 186.33
 AMD    |     32 |   16 |    16 |   1090 | 166.98
 SPY    |     32 |   16 |    16 |    358 | 565.13
 GOOGL  |     30 |   15 |    15 |   1306 | 157.58
(8 rows)
```

Scalar, correlated and EXISTS subqueries in one statement

```
SELECT u.full_name,
       -- scalar subquery
       (SELECT count(*) FROM trading.trade_order o2
          JOIN trading.account a2 USING (account_id)
         WHERE a2.user_id = u.user_id)      AS orders,
       -- correlated subquery
       (SELECT max(o3.placed_at) FROM trading.trade_order o3
          JOIN trading.account a3 USING (account_id)
         WHERE a3.user_id = u.user_id)      AS last_order
  FROM identity.app_user u
 WHERE EXISTS (SELECT 1 FROM trading.account a
                WHERE a.user_id = u.user_id)
   AND u.role = 'trader' ORDER BY orders DESC;
```

```
       full_name       | orders |          last_order
-----------------------+--------+-------------------------------
 Samrender Singh Saini |     54 | 2026-10-18 22:58:09.641226+00
 Vartika Chaudhary     |     47 | 2026-10-18 22:58:08.626855+00
 Ginni Kaur            |     32 | 2026-10-18 19:57:48.123864+00
 Kumar Paritosh        |     30 | 2026-10-16 18:11:03+00
 Pari Sharma           |     19 | 2026-10-18 22:57:48.198088+00
 Sam Saini             |     16 | 2026-10-18 22:58:08.396372+00
(6 rows)
```

SQL Queries and Outputs — CTE and window functions

A running cash balance depends on every earlier row in the partition

```
WITH cash AS (
    SELECT p.account_id, p.currency, p.amount, e.occurred_at
      FROM ledger.posting p JOIN ledger.journal_entry e USING (entry_id)
     WHERE p.asset_class = 'cash' AND p.account_id = :a_sam)
SELECT occurred_at::date AS on_date, currency, round(amount,2) AS amount,
       round(sum(amount) OVER (PARTITION BY currency ORDER BY occurred_at
                               ROWS UNBOUNDED PRECEDING), 2) AS running_cash,
       rank() OVER (ORDER BY abs(amount) DESC)               AS size_rank
  FROM cash ORDER BY occurred_at LIMIT 8;
```

```
  on_date   | currency |  amount   | running_cash | size_rank
------------+----------+-----------+--------------+-----------
 2026-08-16 | USD      | 500000.00 |    500000.00 |         1
 2026-08-17 | USD      | -19964.69 |    480035.31 |        11
 2026-08-20 | USD      | -19997.70 |    460037.61 |        10
 2026-08-21 | USD      |  -7796.54 |    452241.07 |        28
 2026-08-25 | USD      | -20026.09 |    432214.98 |         6
 2026-08-25 | USD      |  -2049.28 |    430165.71 |        42
 2026-08-28 | USD      |  -2049.34 |    428116.37 |        41
 2026-08-31 | USD      | -12007.16 |    416109.21 |        16
(8 rows)
```

A correlated subquery would make this quadratic. The window frame ROWS UNBOUNDED PRECEDING computes it in one ordered pass.

INSERT, UPDATE and DELETE

Shown together in one transaction, then rolled back

```
BEGIN;
INSERT INTO social.watchlist_item (watchlist_id, instrument_id, note)
SELECT w.watchlist_id, i.instrument_id, 'added for the report'
  FROM social.watchlist w, reference.instrument i
 WHERE w.user_id = :u_sam AND w.is_default AND i.symbol = 'INTC'
RETURNING watchlist_id, instrument_id, note;

UPDATE risk.risk_limit
   SET max_order_notional = max_order_notional * 1.10
 WHERE account_id = :a_sam
RETURNING account_id, round(max_order_notional,2) AS new_cap;

DELETE FROM social.watchlist_item
 WHERE watchlist_id IN (SELECT watchlist_id FROM social.watchlist
                        WHERE user_id = :u_sam)
   AND note = 'added for the report'
RETURNING instrument_id, note;
ROLLBACK;
```

```
 watchlist_id | instrument_id |         note
--------------+---------------+----------------------
          356 |           601 | added for the report
(1 row)

 account_id |  new_cap
------------+------------
        375 | 1100000.00
(1 row)

 instrument_id |         note
---------------+----------------------
           601 | added for the report
(1 row)

-- ROLLBACK issued: none of the three statements above is durable.
```

Why RETURNING

- It is not a convenience. It is the only way to learn a generated identity key without a second round trip that could race another session.
- The ROLLBACK leaves the database exactly as it was found.

The complete order round trip

Submission, matching, settlement and cancellation — all inside the database

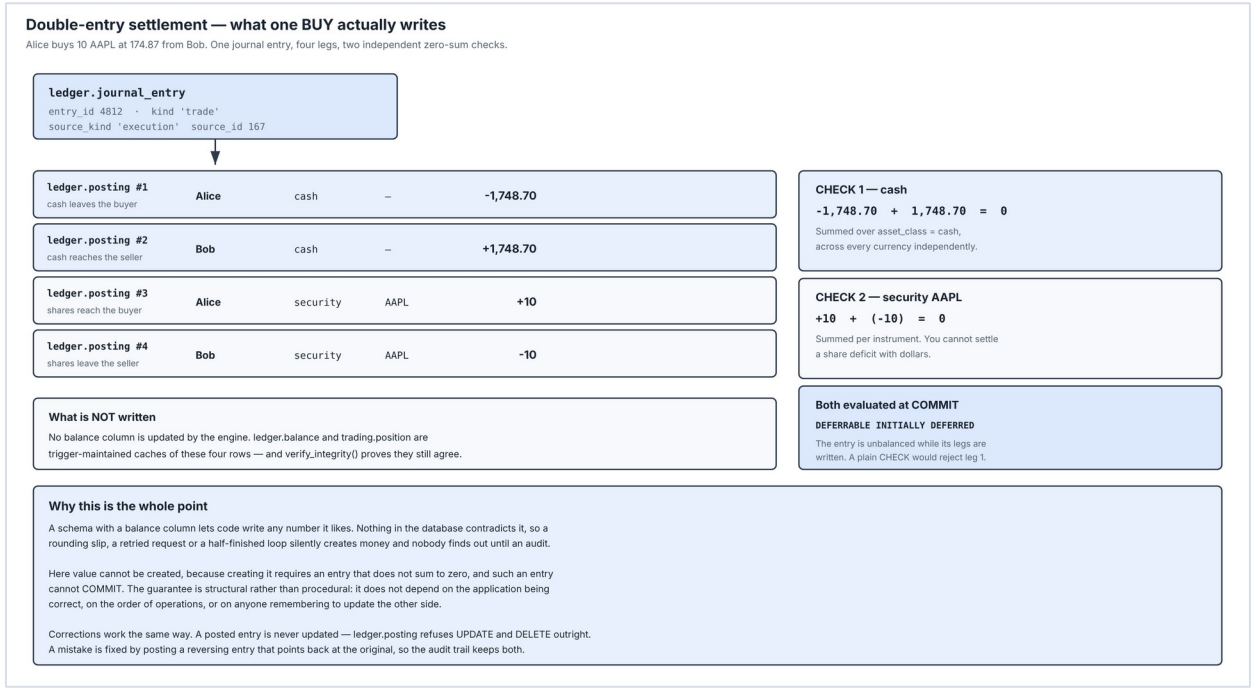
```
-- (a) The Vartika account sends a limit SELL of 10 AAPL, asking 150.00.
--     The book already holds a better bid, so price-time priority
--     fills the aggressor AT THE RESTING PRICE, not at its own limit.
 order_id | status | filled_qty  | avg_fill_price | reject_reason
----------+--------+-------------+----------------+---------------
     9713 | filled | 10.00000000 |   174.73000000 |
(1 row)

-- (b) The Samrender account bids 155.00, below the best ask, so the
--     order rests on the book in status working instead of filling.
 order_id | status
----------+---------
 9714     | working
(1 row)

-- (c) The execution row the engine wrote for (a)
 execution_id | price  |  qty  | aggressor_side | buyer | seller
--------------+--------+-------+----------------+-------+--------
         4049 | 174.73 | 10.00 | sell           |     4 |    376
(1 row)
```

The first order is instructive. It asks to sell at 150.00 and fills at 174.73 — not an error, but price-time priority awarding the aggressor the best resting price on the opposite side of the book, which here was a bid well above the seller's own limit. The second order bids below the best ask, does not cross, and rests on the book in status working.

Settlement — one purchase, four legs, two zero-sum checks




```
 account_id | asset_class | sym  | currency |  amount
------------+-------------+------+----------+----------
          4 | cash        | -    | USD      | -1747.65
        376 | cash        | -    | USD      |  1745.55
          4 | security    | AAPL |          |    10.00
        376 | security    | AAPL |          |   -10.00
          2 | cash        | -    | USD      |     2.10
(5 rows)
```

```
 asset_class | currency | must_be_zero
-------------+----------+--------------
 cash        | USD      |   0.00000000
 security    |          |   0.00000000
(2 rows)
```

Summed per asset class, the legs are exactly zero. That is the condition the deferred constraint checks at COMMIT.

Operations the database refuses

A system that only demonstrates success has demonstrated half its behaviour

```
-- (a) Selling stock the account does not hold, short selling off
  status  |                                  reject_reason
----------+-------------------------------------------------------------------------
 rejected | cannot sell 5 NVDA: you hold 0. Short selling is not enabled on this account.
(1 row)

-- (b) A buy far beyond the single-order notional cap
  status  |                       reject_reason
----------+-------------------------------------------------------------
 rejected | order value 20000000 USD exceeds your limit of 1000000 USD
(1 row)

-- (c) A limit price outside the instrument price band
  status  |                  reject_reason
```

Short-sale gate

- The account does not hold the stock and short selling is off.
- risk.pretrade_check compares the order against holdings and open sell exposure before anything reaches the book.

Notional cap

- Per-account single-order limit, held in risk.risk_limit.
- Converted into the account's base currency first, so the cap means the same thing in INR and in USD.

Price band

- Per-instrument percentage band around the reference price.
- A rejected order is a normal business outcome, so the reason is RETURNED to the caller, not raised as an exception.

Transaction Control — a constraint that fires at COMMIT

```
-- The INSERT below is accepted: the check is DEFERRED.
 posting_id | account_id |    amount
------------+------------+---------------
      24876 |        375 | 1000.00000000
(1 row)

-- COMMIT now runs the constraint trigger:
psql:q09_defer.sql:13: ERROR:  ledger entry 5825 has 1 posting(s): double-entry requires at least two
HINT:  value must move between two named accounts
CONTEXT:  PL/pgSQL function ledger.assert_entry_balanced() line 11 at RAISE
```

What the listing shows

- The INSERT succeeds — the check is DEFERRED.
- The COMMIT fails, and the whole transaction is discarded.
- The entry had one leg; double entry requires at least two, and the legs must sum to zero per asset class.

Why this matters

- The ledger balances not by discipline in the application, and not by a nightly reconciliation job, but by a rule the database will not allow a transaction to violate.
- Any future code that writes to ledger.posting inherits the guarantee.

Transaction Control — COMMIT, ROLLBACK and SAVEPOINT

```
BEGIN;
INSERT INTO social.watchlist (user_id, name)
VALUES (:u_sam, 'Report A') RETURNING watchlist_id, name;
SAVEPOINT after_first;
INSERT INTO social.watchlist (user_id, name)
VALUES (:u_sam, 'Report B') RETURNING watchlist_id, name;
ROLLBACK TO SAVEPOINT after_first;
-- only Report A remains pending
SELECT name FROM social.watchlist
 WHERE user_id = :u_sam AND name LIKE 'Report%';
ROLLBACK;
-- the demonstration leaves nothing behind
SELECT count(*) AS report_rows_left
  FROM social.watchlist WHERE name LIKE 'Report%';
```

```
 watchlist_id |   name
--------------+----------
          368 | Report A
(1 row)

 watchlist_id |   name
--------------+----------
          369 | Report B
(1 row)

-- Rolled back to the savepoint: only Report A remains pending.
   name
----------
 Report A
(1 row)

-- Outer ROLLBACK: the demonstration leaves nothing behind.
 report_rows_left
------------------
                0
(1 row)
```

BEGIN	Opens the unit of work; the deferred constraint is now pending
SAVEPOINT name	Marks a point the transaction can be returned to
ROLLBACK TO SAVEPOINT	Undoes later statements, keeping the transaction open
COMMIT	Runs every deferred constraint, then makes the work durable
ROLLBACK	Discards the whole unit of work, leaving no trace

Advanced — Procedures and Functions

84 routines. They are not a convenience layer; they are where the rules live.

Routine	Language	Responsibility
`engine.submit_order`	PL/pgSQL	Validate, risk-check, persist, lock the book, match
`engine.match_order`	PL/pgSQL	The price-time priority matching loop
`engine.settle_fill`	PL/pgSQL	Write the execution and post the journal entry
`engine.elect_stops`	PL/pgSQL	Arm and elect stop and trailing-stop orders
`ledger.post_entry`	PL/pgSQL	Write an entry and its legs as one atomic unit
`ledger.verify_integrity`	PL/pgSQL	Re-derive every balance and report drift
`risk.pretrade_check`	PL/pgSQL	Buying power, short gate, cap, band, halts
`market.candles`	SQL	Any chart interval from the one-minute base table
`market.candles_with_indicators`	SQL	SMA, EMA, Bollinger bands and RSI by window
`analytics.performance_stats`	SQL	Return, drawdown, Sharpe ratio, win rate
`surveil.run_all_scans`	PL/pgSQL	Four market-abuse detections
`ops.health_check`	SQL	Evaluate the eighteen structural invariants
`ops.end_of_day`	PROCEDURE	Expire, snapshot, refresh, roll partitions

ops.end_of_day is a PROCEDURE rather than a function for a specific reason: a procedure may issue COMMIT, and the end-of-day run is a sequence of independent steps that should not be undone as a group if the last of them fails.

Advanced — the matching loop

Price-time priority expressed as an ORDER BY that an index can serve

```
-- Find the single best contra order.
-- The ORDER BY here *is* price-time priority, and it is served
-- directly by ix_book_bids / ix_book_asks with no sort step.
-- SKIP LOCKED is deliberately NOT used: skipping a locked order
-- would violate time priority by letting a later order jump the
-- queue, so we block and preserve fairness instead.
SELECT c.order_id, c.limit_price, c.leaves_qty, c.account_id
  INTO v_contra
  FROM trading.trade_order c
 WHERE c.instrument_id = v_o.instrument_id
   AND c.side   = 'sell'
   AND c.status IN ('working','partially_filled')
   AND NOT c.stop_armed
   AND c.limit_price IS NOT NULL
   AND c.leaves_qty  > 0
   AND (v_limit IS NULL OR c.limit_price <= v_limit)
 ORDER BY c.limit_price ASC, c.seq_no ASC
 LIMIT 1
 FOR UPDATE;
```

The index that serves it

- ix_book_asks is a PARTIAL B-tree over (instrument_id, limit_price, seq_no) WHERE the order is still resting.
- It covers only the live book, so it stays small no matter how much history accumulates.

FOR UPDATE, not SKIP LOCKED

- SKIP LOCKED is the usual advice for a queue. An order book is not a queue: skipping a locked order lets a later order jump ahead, which is exactly the fairness the venue exists to guarantee.

The serialisation point

- pg_advisory_xact_lock(hashtext('apex_book'), instrument_id)

Advanced — Triggers

Twenty on the base tables, in four groups

```
        table_name         |          trigger_name           |    kind
---------------------------+---------------------------------+----------
 identity.app_user         | trg_audit_user                  | row
 identity.app_user         | trg_user_default_watchlist      | row
 ledger.journal_entry      | trg_journal_immutable           | row
 ledger.posting            | trg_posting_balanced            | constrain
 ledger.posting            | trg_posting_immutable           | row
 ledger.posting            | trg_posting_syncs_position      | row
 ledger.posting            | trg_posting_updates_balance     | row
 social.copy_subscription  | trg_no_self_copy                | row
 surveil.alert             | trg_audit_alert                 | row
 trading.account           | trg_account_default_limits      | row
 trading.account           | trg_audit_account               | row
 trading.execution         | trg_block_self_match            | row
 trading.execution         | trg_execution_enqueues_copies   | row
 trading.trade_order       | trg_audit_order_material        | row
 trading.trade_order       | trg_audit_order_write           | row
(15 rows)
```

Maintain derived state

- trg_posting_updates_balance, trg_posting_syncs_position

Enforce an invariant

- trg_posting_balanced (deferred), trg_block_self_match, trg_no_self_copy

Make a table append-only

- trg_journal_immutable, trg_posting_immutable

Write the audit trail

- trg_audit_user, trg_audit_account, trg_audit_order_write

Advanced — Views and Materialized Views

and the security-mode defect that none of them reported

```
       object_name        |  kind   | runs_as
--------------------------+---------+---------
 mv_instrument_daily      | matview | definer
 mv_strategy_leaderboard  | matview | definer
 quote_enriched           | view    | definer
 v_account_summary        | view    | invoker
 v_alert_queue            | view    | invoker
 v_blotter                | view    | invoker
 v_book_depth             | view    | definer
 v_fx_board               | view    | definer
 v_open_orders            | view    | invoker
 v_portfolio              | view    | invoker
 v_watchlist              | view    | invoker
(11 rows)
```

What the runs_as column records

- A view executes with the privileges of its OWNER, and a table owner is exempt from Row Level Security on its own tables.
- Every view over user data was therefore returning every user’s rows while the policies beneath it read as correct.

The fix, and the guard

- WITH (security_invoker = true) — the view now executes as the caller.
- ops.assert_rls_intact() fails if a view over a policy-protected table is ever created without it.
- The five definer-rights views expose only reference and market data, which is not user-specific.

Advanced — Row Level Security

Row Level Security — authorisation that a wrong query cannot bypass

The same SQL text, two sessions, two answers. The restriction is on the table, not on the statement.

```
SELECT symbol, quantity, market_value FROM analytics.v_portfolio ORDER BY 3 DESC;
```

```
SET apex.current_user_id = 342
```

Samrender's session

V	171.00	51,057.18
NFLX	48.00	33,365.28
TSLA	58.00	32,557.14
INTC	684.00	30,831.85

```
SET apex.current_user_id = 343
```

Vartika's session

SPY	87.00	53,696.40
QQQ	53.00	35,671.65
MSFT	30.00	22,536.60
JPM	71.00	17,750.00

The policy

```
CREATE POLICY p_position_own ON trading.position
  FOR SELECT USING (
    identity.current_user_is_privileged()
    OR account_id IN (SELECT a.account_id
                      FROM trading.account a
                     WHERE a.user_id =
                           identity.current_user_id()));
```

The API connects as apex_app, which owns no table —
a table owner is exempt from its own policies.

The defect this caught

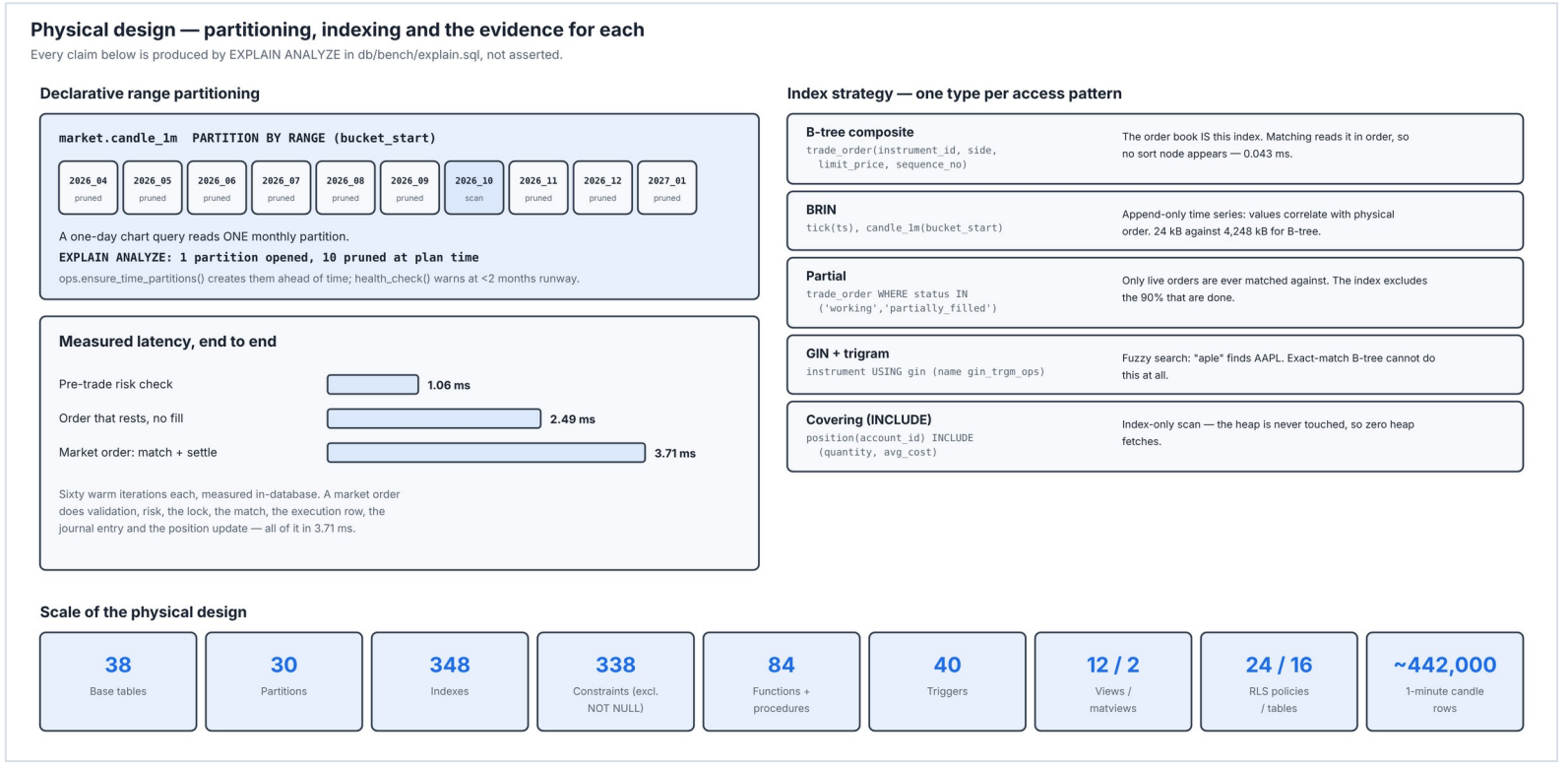
Every view over user data was returning every user's rows, while the policies beneath it read as correct. A view executes as its OWNER, and the owner is exempt from RLS on its own tables.

Fix: WITH (security_invoker = true), so the view runs as the caller.

Guard: ops.assert_rls_intact() fails if a view over a policy-protected table is ever created without it. The repair is one line; the invariant is what stops it coming back.

Connected as apex_app, the unprivileged role the API uses. A session can see exactly one account, and no query it can write will widen that.

Advanced — Partitioning and the index strategy




```
                                                
```

Advanced — Market-abuse surveillance

Market-abuse surveillance — four detections, each one analytical SQL

No rules engine, no stream processor. Each rule is a query over the order and execution history.

WASH_TRADE A self-join	Self-join of execution through both order sides back to the owning user	critical
SPOOFING Window functions	Cancel-to-fill ratio per account per window, ranked against the venue	high
MOMENTUM_IGNITION LAG / LEAD	Burst of same-side aggression followed by a reversal within minutes	high
VOLUME_ANOMALY Statistical outlier	Today's volume against a 20-day mean and standard deviation	medium

The wash-trade query, in outline

```
SELECT ba.user_id, e.instrument_id,
       count(*)                 AS trade_count,
       sum(e.quantity)          AS wash_qty,
       sum(e.price*e.quantity)  AS wash_notional,
       array_agg(e.execution_id ORDER BY e.executed_at)
  FROM trading.execution e
  JOIN trading.account ba ON ba.account_id = e.buy_account_id
  JOIN trading.account sa ON sa.account_id = e.sell_account_id
 WHERE e.executed_at > now() - p_lookback
   -- Same human on both sides, through one account or two.
   AND ba.user_id = sa.user_id
 GROUP BY ba.user_id, e.instrument_id
```

The alert it raised

rule_code	WASH_TRADE
severity	critical
score	60.0
trade_count	6
washed_qty	1,200.00
washed_notional	54,096.00
state	open

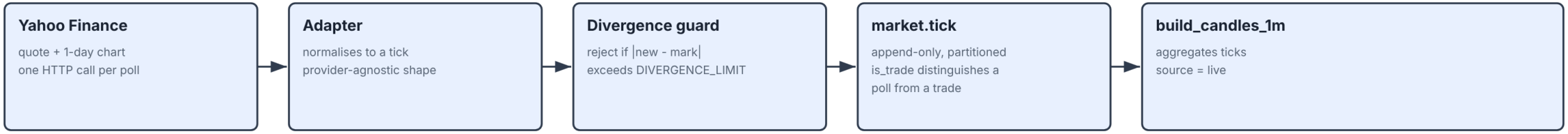
Evidence is stored as JSONB and indexed with GIN (jsonb_path_ops), so an alert can be interrogated months later. A unique dedupe_key makes every scan idempotent.

Evidence is stored as JSONB and indexed with GIN, so an alert raised today can still be interrogated months later.

Market data — and the two guards added after it broke

Market-data path — and the two guards that were added after it broke

Each guard exists because a specific defect reached the running system.



Defect 1 — a wrong price for an unprimed symbol

The worker primed its price profile once, at startup. A symbol added later fell through to a hash-derived pseudo-profile, and ADA-USD was quoted at $57 instead of $0.39.
Fix: lazy per-symbol priming, so a symbol is primed the first time it is polled.

Defect 2 — live polling overwrote backfilled history

build_candles_1m() rebuilt every bucket in its window from ticks, including the months that had been backfilled from the provider, destroying them.
Fix: migration 023 added candle_1m.source. The roller may only overwrite a bucket it wrote itself; history marked backfill is untouchable.

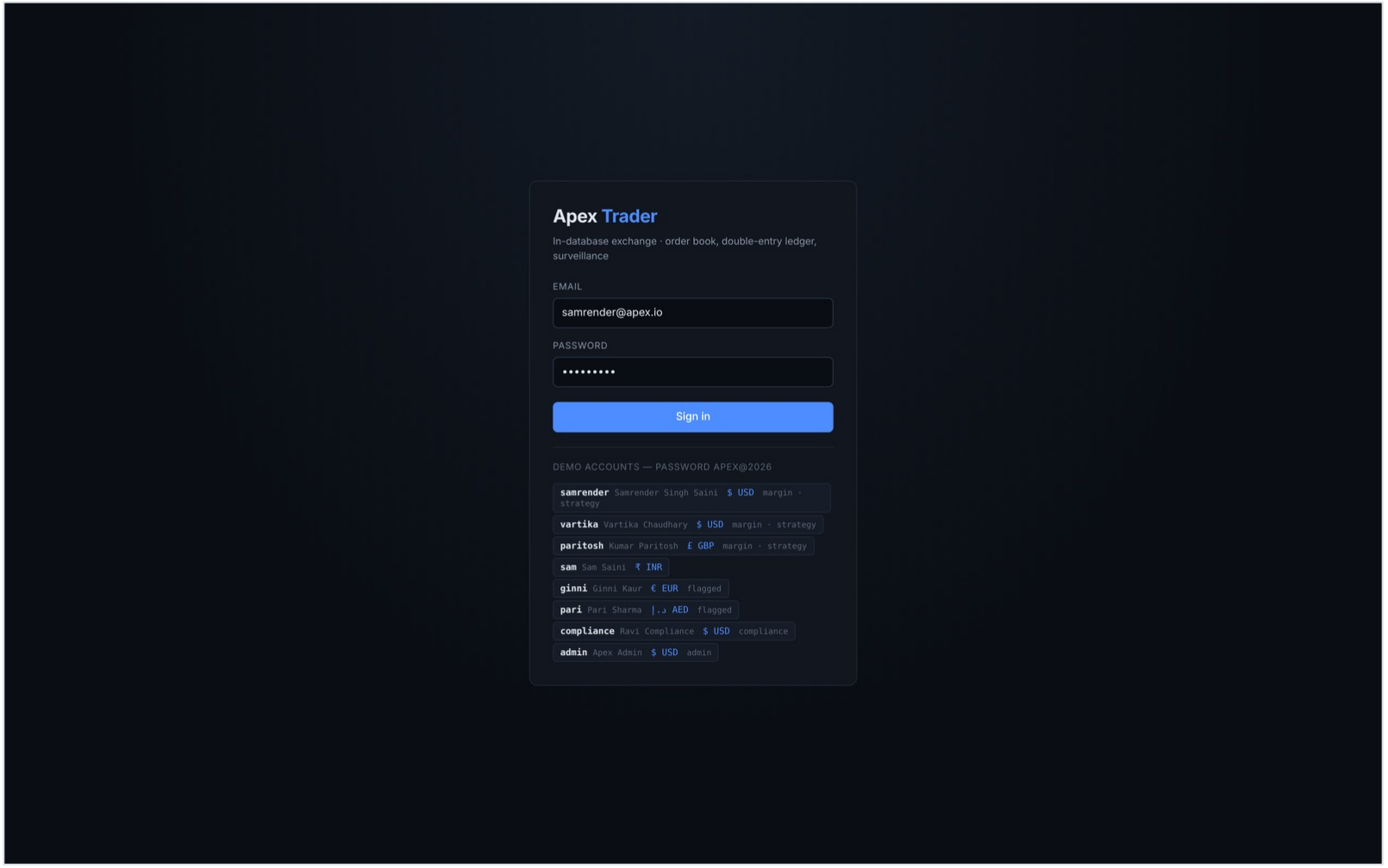
The invariant that now proves both fixes

```
history_is_marked_backfill   loaded history is protected from the tick roller
marks_carry_no_volume        reference marks are excluded from volume
bands_contain_their_books    every price band still contains its own order book
```

Each is one row of ops.health_check(). A regression stops being a silent wrong number and becomes a failing check on the compliance console.

Both defects produced wrong data rather than an error message. Both are now covered by an invariant that fails loudly.

Application — authentication



Six named demonstration accounts, a compliance officer and an administrator. Passwords are hashed by pgcrypto with a per-row salt; identity.authenticate compares the hash, counts failures and applies a lockout, all in the database.

Application — the trading terminal



Candlestick chart with moving averages, Bollinger bands, volume and RSI, a live depth-of-market ladder, and the order ticket. The indicators are returned by market.candles_with_indicators, not computed in the browser.

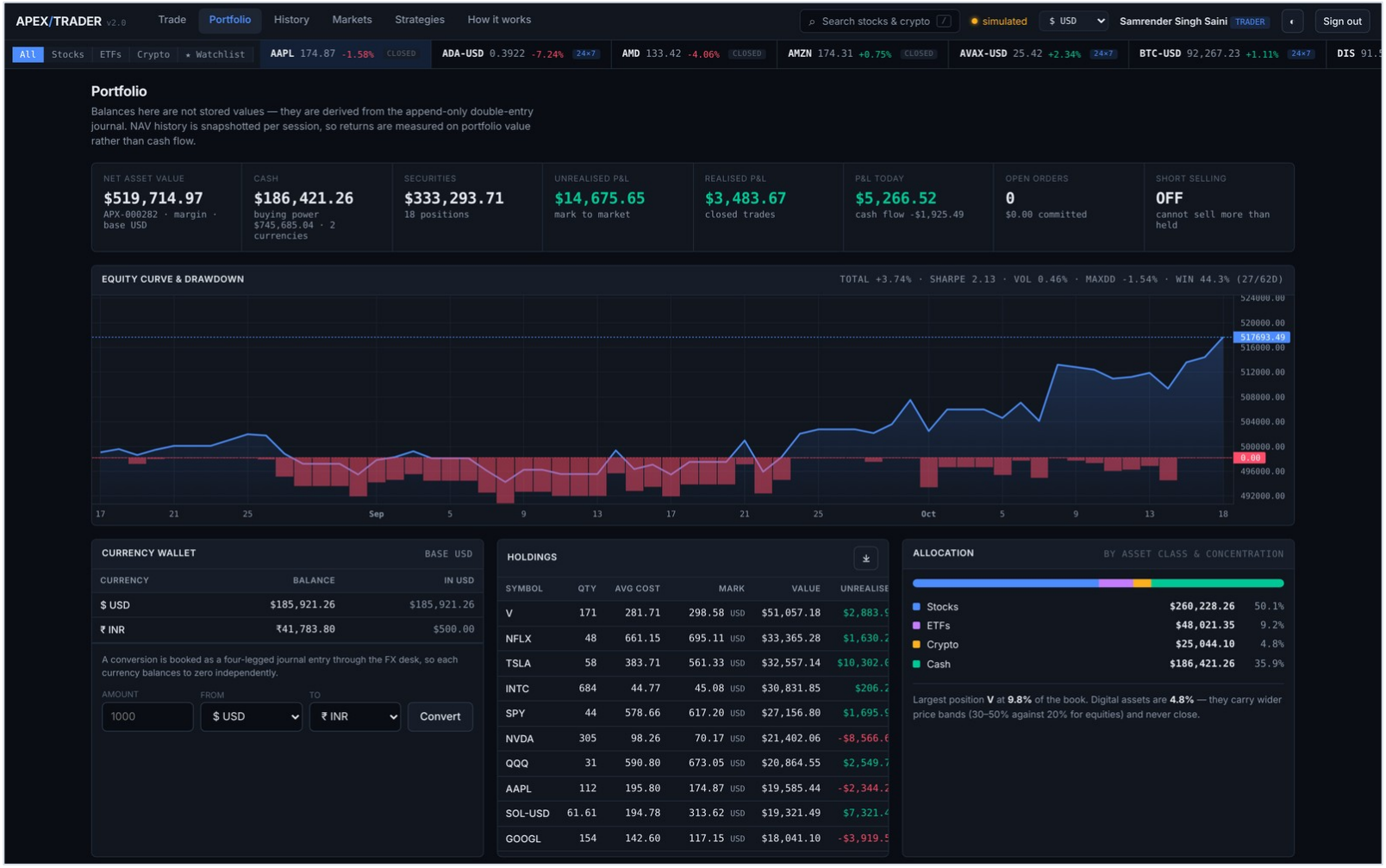
Application — positions and the ledger

Open orders | **Positions** 18 | Fills | Time & sales | Ledger | ★ Watchlist | 🔔 Alerts

SYMBOL	NAME	QTY	AVG COST	MARK	VALUE	UNREALISED	%	REALISED	% OF BOOK
V	Visa Inc.	171	281.71	298.58	$51,057.18	$2,883.98	+5.99%	-$39.86	15.3%
NFLX	Netflix Inc.	48	661.15	695.11	$33,365.28	$1,630.26	+5.14%	$0.00	10.0%
TSLA	Tesla Inc.	58	383.71	561.33	$32,557.14	$10,302.07	+46.29%	$692.41	9.8%
INTC	Intel Corporation	684	44.77	45.08	$30,831.85	$206.20	+0.67%	$692.12	9.3%
SPY	SPDR S&P 500 ETF Trust	44	578.66	617.20	$27,156.80	$1,695.95	+6.66%	$126.72	8.2%
NVDA	NVIDIA Corporation	305	98.26	70.17	$21,402.06	-$8,566.66	-28.59%	$0.00	6.4%
QQQ	Invesco QQQ Trust	31	590.80	673.05	$20,864.55	$2,549.75	+13.92%	$51.54	6.3%
AAPL	Apple Inc.	112	195.80	174.87	$19,585.44	-$2,344.21	-10.69%	$0.00	5.9%
SOL-USD	Solana	61.61	194.78	313.62	$19,321.49	$7,321.49	+61.01%	$0.00	5.8%
GOOGL	Alphabet Inc.	154	142.60	117.15	$18,041.10	-$3,919.58	-17.85%	$0.00	5.4%

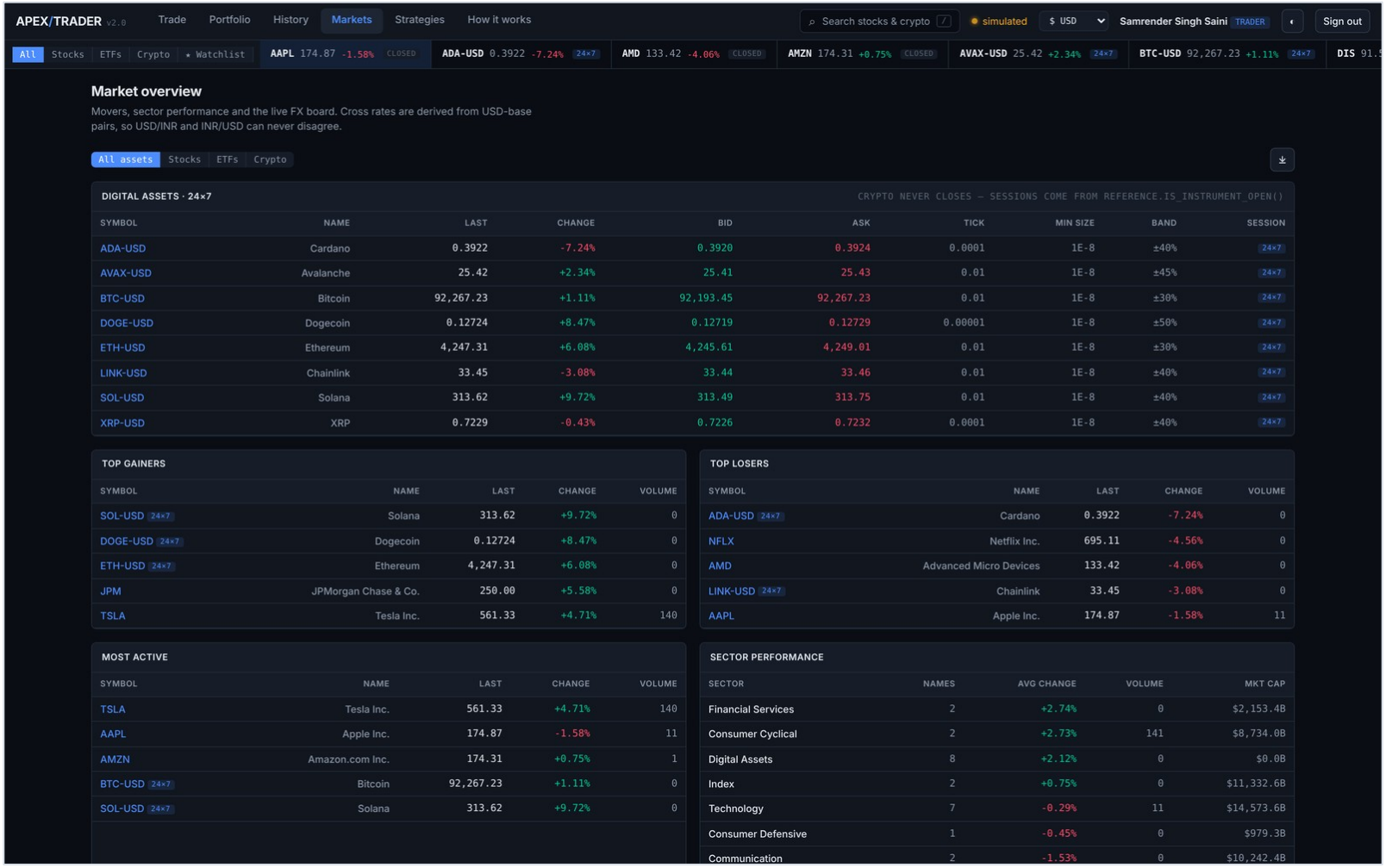
The positions tab shows what the account holds; the ledger tab shows every journal entry that produced those holdings. The first is derived from the second.

Application — portfolio and performance



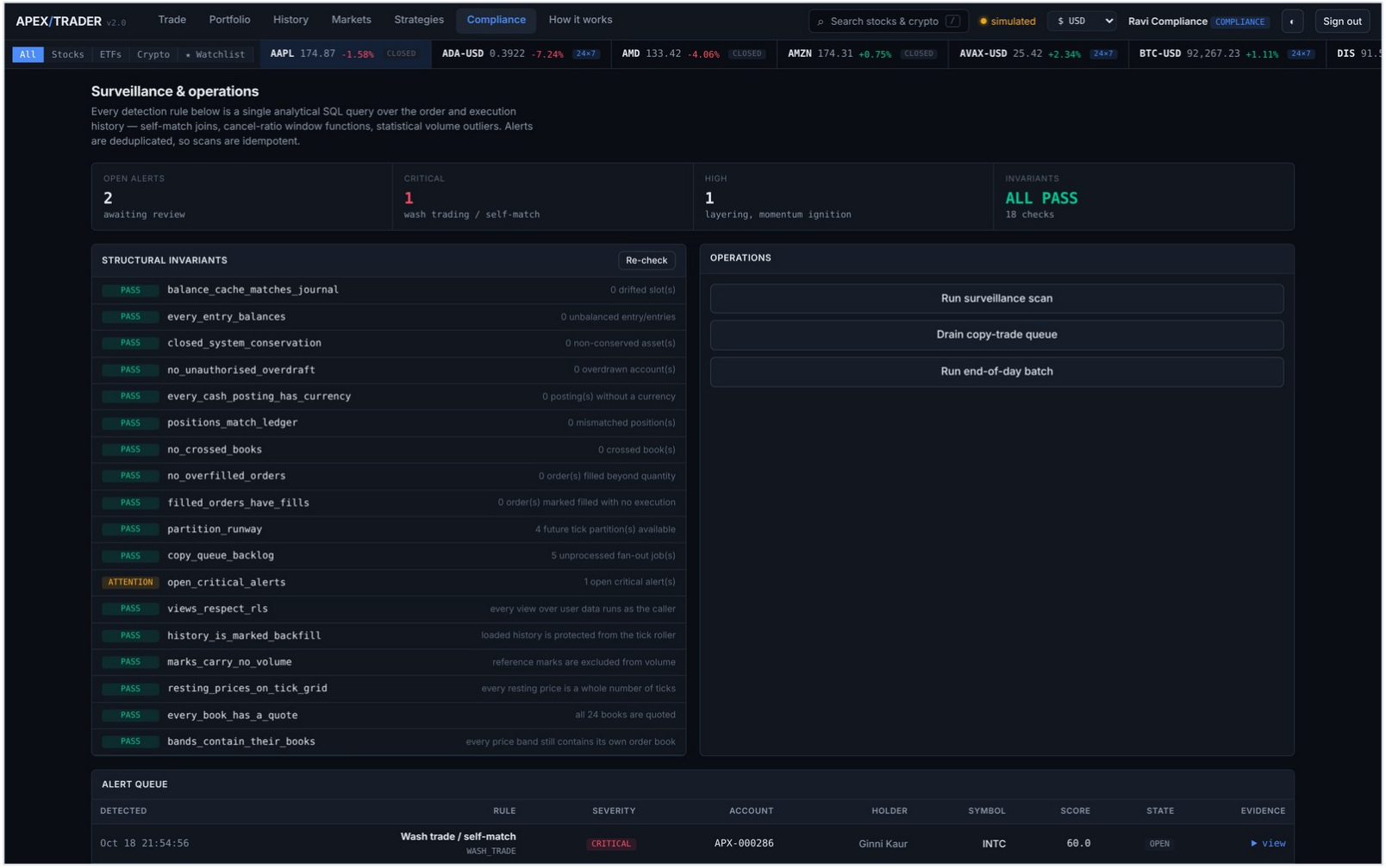
Net asset value curve, allocation and performance statistics. Return, drawdown, Sharpe ratio and win rate come from analytics.performance_stats in one statement.

Application — markets, equities and crypto



Crypto instruments carry trades_24x7, so reference.is_instrument_open returns true at all hours while an equity is closed outside its session. The distinction is reference data, not a conditional in the application.

Application — the compliance console



The eighteen invariants with their current status, the open surveillance alerts with the evidence behind each, and the operational actions. Reachable only by a user whose role in identity.app_user is compliance or admin.

Testing and Results — concurrency

Concurrency — the proof, not the claim

An order book is a contended resource. The failure mode is the double fill: two buyers read the same resting order and both match it.

The harness db/test/concurrency.sh

```
One resting SELL order:  3,000 shares @ 182.00
12 independent psql sessions, 25 orders each
Total demand 3,000 shares against 3,000 available
```

Demand exactly equals supply, so any overfill and any underfill are both visible in the result.

The mechanism

```
PERFORM pg_advisory_xact_lock(
        hashtext('apex_book'), p_instrument_id);
```

One book at a time; different instruments never contend. The lock is transaction-scoped, so it is released by COMMIT or ROLLBACK and cannot leak.

The matching SELECT takes FOR UPDATE and NOT SKIP LOCKED: skipping a locked order would let a later order jump the queue.

The result

3,000	300	3,000	0	0	0.00000000	0.00000000
shares offered	fills	shares traded	overfilled orders	rejected orders	shares created or destroyed	cash created or destroyed

And the invariants still hold afterwards

```
balance_cache_matches_journal     0 drifted slot(s)
every_entry_balances              0 unbalanced entry/entries
closed_system_conservation        0 non-conserved asset(s)
no_unauthorised_overdraft         0 overdrawn account(s)
every_cash_posting_has_currency   0 posting(s) without a currency
```

CONCURRENCY TEST PASSED

12 sessions, no double-fill, ledger exact

Total demand exactly equals available supply, so both an overfill and an underfill would be visible in the result.

Testing and Results — four independent suites

They check different things on purpose

Suite	What it exercises	Result
`db/test  (5 files)`	Ledger, engine, risk, currency and crypto rules	**150 assertions passed**
`db/test/concurrency.sh`	12 sessions × 25 orders against one resting order	**Passed, no double fill**
`scripts/api_smoke.py`	Every REST path, authorisation and error case	**118 checks passed**
`scripts/ui_check.py`	Browser rendering of all seven screens	**51 checks passed**
`ops.health_check()`	Eighteen structural invariants, on live data	**18 of 18 reporting**

```
  suite   | passed | failed
----------+--------+--------
 crypto   |     31 |      0
 currency |     32 |      0
 engine   |     32 |      0
 ledger   |     16 |      0
 risk     |     39 |      0
(5 rows)

all 150 assertions passed
```

```
          check_name             |  status   |                  deta
---------------------------------+-----------+----------------------
 open_critical_alerts            | ATTENTION | 1 open critical alert
 balance_cache_matches_journal   | PASS      | 0 drifted slot(s)
 bands_contain_their_books       | PASS      | every price band stil
 closed_system_conservation      | PASS      | 0 non-conserved asset
 copy_queue_backlog              | PASS      | 5 unprocessed fan-out
 every_book_has_a_quote          | PASS      | all 24 books are quot
 every_cash_posting_has_currency | PASS      | 0 posting(s) without
 every_entry_balances            | PASS      | 0 unbalanced entry/en
 filled_orders_have_fills        | PASS      | 0 order(s) marked fil
```

The one ATTENTION row is the surveillance scanner doing its work: a wash trade was found and the alert is open.

What was learnt

Three defects, one shape: no error message, only a wrong answer

Every view bypassed Row Level Security

A view runs as its owner, and a table owner is exempt from its own policies. The policies were correct and the data was exposed anyway.

Fix: security_invoker. Guard: ops.assert_rls_intact().

The candle roller destroyed backfilled history

build_candles_1m rebuilt every bucket in its window from ticks — correct for live minutes, destructive for months loaded from the provider.

Fix: candle_1m.source. Guard: history_is_marked_backfill.

The test suite reported green while checking less

A type mismatch inside one helper raised an error that psql swallowed because ON_ERROR_STOP was not set.

Fix: the cast. Guard: test.sh now fails on any error psql reports.

In all three cases the correctness became something the database asserts, rather than something the authors remember.

Conclusion

Apex Trader is a complete, working trading system in which the database is not a store behind the application but the application itself. The service tier validates request shape and emits JSON; every rule that matters is below it, where a second client is subject to it without reimplementing any of it.

In-database matching with price-time priority

The book is correct under concurrency because the database serialises access to it

Double-entry ledger with derived balances

A balance cannot drift from its history, because the history is the balance

Deferred zero-sum constraint

Value can move between accounts but can never be created or destroyed

Row Level Security with security_invoker views

A forgotten predicate returns nothing rather than another trader's rows

Declarative partitioning and a deliberate index strategy

The chart query opens one month of 442,000 rows in well under a millisecond

Eighteen invariants checked on live data

Defects that produce no error message are still detected

Future Enhancement

Enhancement	Approach
Logical replication to a read replica	Route analytics and chart reads to a replica; the write path is already one function call
Continuous aggregates for candles	Replace the scheduled rollup with an incrementally maintained aggregate
Options and futures	Reference data already models tick size, lot size and contract metadata; the ledger needs a third asset class
Settlement cycles (T+1)	The journal already carries occurred_at; a settlement date and a pending-settlement account complete it
Order-book depth beyond top of book	The resting-order index already supports it; only aggregation and transport are missing
Partition detachment to cold storage	ops.detach_old_partitions exists; the remaining work is archival and restore
Formal isolation testing with pgbench	Extend the concurrency proof to serialisation anomalies under REPEATABLE READ

References

[1] Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, 7th ed., Pearson, 2016.

[2] Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, 7th ed., McGraw-Hill, 2019.

[3] Date, C. J., An Introduction to Database Systems, 8th ed., Addison-Wesley, 2003.

[4] Codd, E. F., "A Relational Model of Data for Large Shared Data Banks", CACM 13(6), 1970.

[5] Codd, E. F., "Recent Investigations into Relational Data Base Systems", IFIP Congress, 1974.

[6] Kleppmann, M., Designing Data-Intensive Applications, O'Reilly, 2017.

[7] PostgreSQL 16 Documentation — Row Security Policies, Table Partitioning, CREATE TRIGGER, SET CONSTRAINTS, Advisory Locks, BRIN and GIN indexes, pgcrypto.

[8] Harris, L., Trading and Exchanges: Market Microstructure for Practitioners, OUP, 2003.

[9] Johnson, B., Algorithmic Trading and DMA, 4Myeloma Press, 2010.

[10] ISO 4217 Currency codes; ISO 6166 ISIN. FIX Protocol Specification 4.4.

[11] SEBI, Master Circular on Surveillance of Securities Market, 2023.

[12] FastAPI, asyncpg and TradingView Lightweight Charts documentation.

[13] Yahoo Finance market data endpoints, accessed October 2026.

Thank you

Apex Trader — BCSE302P Database Systems Lab

Kumar Paritosh · Samrender Singh Saini · Vartika Chaudhary

Under the guidance of Prof. Nithya N S